Lattice-Holo OS Whitepaper

Lattice Semantic Transport Protocol v0.1

Status: Draft specification whitepaper
Protocol name: Lattice Semantic Transport Protocol
Short name: LSTP
Project vision: Lattice-Holo OS
Version: v0.1
Core principle: One meaning. Many carriers. Always inspectable.

Abstract

The Lattice Semantic Transport Protocol, or LSTP, is a proposed semantic communication protocol for human-AI and AI-AI interaction. It is designed to reduce the inefficiencies of ordinary natural language while preserving interpretability, auditability, safety, and human usability.

The central idea is that communication should be represented first as a canonical semantic packet rather than as ordinary text, speech, images, or vectors. Text, graph structures, radial visual interfaces, audio signals, and AI-native representations are treated as carriers of the same underlying packet. The packet itself is the source of meaning.

LSTP v0.1 defines a draft architecture built around the **Octad Packet**, a structured representation containing pragmatics, atoms, relations, context, confidence, permissions, evidence, and output requirements. This whitepaper presents the problem statement, design evolution, synthesis of Lattice, HoloSemantics, and LΩ concepts, final architecture, protocol overview, safety model, limitations, and implementation roadmap.

This document is a design whitepaper, not a completed standard. Its purpose is to make the protocol implementable enough for a first parser, packet schema, examples, conformance tests, and experimental radial interface.

---

1. Problem Statement

Human language is flexible, expressive, and culturally rich, but it is not optimized for high-density communication with AI systems. Ordinary language has several practical limitations in AI-era communication:

1. **Linearity**
   Human language usually expresses ideas sequentially. Complex thoughts involving entities, constraints, assumptions, uncertainty, evidence, permissions, and desired output must be compressed into a one-dimensional stream of words.

2. **Ambiguity**
   Natural language often leaves intent, scope, evidence, or assumptions implicit. A request such as

> “analyze the report” may leave unclear what kind of analysis is needed, what criteria matter, how much detail is expected, and whether the AI is allowed to take action.

3. **Repetition of Context**
   Conversations often require repeated references to earlier messages, prior documents, previous assumptions, or shared goals. This creates unnecessary cognitive and computational overhead.

4. **Hidden Uncertainty**
   Natural language frequently hides degrees of confidence. Phrases such as “probably,” “maybe,” or “I think” are imprecise and difficult to standardize.

5. **Weak Permission Signaling**
   Ordinary language does not reliably separate requests for advice, drafts, previews, simulations, real-world actions, or irreversible execution. This is especially important when AI systems can send messages, change files, execute code, or coordinate with other agents.

6. **Poor Auditability in Compressed Representations**
   Pure vector or embedding-based communication may be efficient for machines, but it is difficult for humans to inspect, debug, validate, or challenge.

The goal of LSTP is not to replace natural language in all human contexts. It is not intended to replace poetry, storytelling, emotional conversation, law, culture, or ordinary speech. Its purpose is narrower:

> LSTP aims to make instruction, analysis, reasoning, coordination, and AI-mediated action more compact, structured, auditable, and safe.

---

2. Design Goals

LSTP v0.1 is guided by the following design goals.

2.1 High Information Density

A short expression should be able to encode intent, target, operation, constraints, uncertainty, output format, and permission state.

Example:

```
!analyze @sales[-4Q] :: cmp(region) + detect(anomaly) + infer(cause~)
{mode=readonly, assume:market_stable}
-> brief@z2
%0.82
; ctx↑0
```

This expression encodes a command, target, time scope, operations, assumption, permission mode, output format, zoom level, confidence threshold, and context reference.

2.2 Human Usability

The protocol must work on ordinary keyboards before it depends on AR, spatial computing, neural interfaces, or specialized audio tools. The text layer is therefore central to v0.1.

2.3 Inspectability

Every meaningful packet must be expandable into a human-auditable representation. Vectors and embeddings may assist memory and retrieval, but they must not replace symbolic structure.

2.4 Multimodal Extensibility

The same semantic packet should be representable through multiple carriers:

- Lattice text
- JSON packet
- Semantic graph
- Radial visual interface
- Audio or haptic status signal
- AI-native symbolic graph with optional vector attachments

2.5 Safety by Design

Permissions, action modes, assumptions, uncertainty, and evidence must be first-class fields, not informal additions.

2.6 Recoverable Compression

The protocol should compress expression, not meaning. A short packet should remain expandable, inspectable, and challengeable.

---

3. Design Evolution

LSTP emerged from the synthesis of three complementary conceptual directions: **Lattice**, **HoloSemantics**, and **LΩ**.

These names refer to proposal families within the design discussion, not to finalized external standards.

3.1 Lattice

Lattice contributed the practical symbolic layer.

Its strengths were:

- compact text syntax;
- explicit intent markers;
- target and operation notation;
- constraints and output control;

- uncertainty markers;
- permission modes;
- zoom levels;
- human readability;
- copyability, searchability, and version control compatibility.

A basic Lattice expression follows this pattern:

```
<intent><action> @<target>[<scope>] :: <operations>
{constraints}
-> <output>@z<n>
%<confidence>
; metadata
```

Example:

```
!review @contract :: risks + obligations + ambiguities
{mode=readonly}
-> memo@z3
```

Lattice is therefore the primary human-auditable serialization of the protocol.

3.2 HoloSemantics

HoloSemantics contributed the multimodal and spatial interface vision.

Its strongest contribution was the idea that semantic packets need not only be typed. In a spatial computing environment, a user could author meaning by manipulating visual objects:

- a central node represents the target;
- operation nodes are dragged onto the target;
- confidence is represented through opacity or border strength;
- context depth is represented through spatial depth;
- zooming expands semantic detail;
- audio or haptic signals communicate urgency, completion, confidence, or required approval.

The major correction applied to this idea is that visual and audio forms should not be treated as the sole source of meaning. Instead, they are carriers, renderers, or authoring interfaces for the canonical packet.

Thus, radial UI is bidirectional:

```
Lattice text → canonical packet → radial visualization
radial manipulation → canonical packet → Lattice text
```

3.3 LΩ

LΩ contributed protocol-oriented concepts:

- context stack;
- stack references such as `↑n`;
- recursive macros;
- semantic fingerprints;
- graph-based structure;
- AI-native exchange;
- separation between atoms, relations, and intent.

The key refinement is that AI-to-AI communication should not rely on pure vectors alone. Instead, it should use symbolic graph packets with optional vector attachments.

A corrected AI-native packet should therefore look like:

```
{
  "symbolic": {
    "intent": "compare",
    "targets": ["H1", "H2"],
    "operation": "evidence_fit"
  },
  "vector_refs": {
    "H1": "vec://project/H1/9382",
    "H2": "vec://project/H2/4410"
  },
  "context_ref": "ctx_92BC"
}
```

This preserves speed and semantic retrieval while maintaining auditability.

---

4. Synthesis

The synthesis of Lattice, HoloSemantics, and LΩ produces the core architecture of LSTP:

> One canonical semantic packet, many carriers, always inspectable.

The final system is not merely a language. It is a semantic transport layer.

The roles are:

Component	Primary contribution
Lattice	Typable symbolic syntax and audit layer
HoloSemantics	Radial, spatial, and sensory interface layer
LΩ	Context stack, graph structure, macros, and AI-native protocol concepts

Component	Primary contribution
LSTP	Unified semantic packet protocol

The final distinction is between **meaning** and **carrier**.

- Meaning lives in the canonical packet.
- Text is a human-auditable carrier.
- Graphs are machine-reasoning carriers.
- Radial UI is a visual and spatial authoring carrier.
- Audio is a status and attention carrier.
- Vectors are assistive memory and similarity carriers.

The packet is not necessarily “truth.” It may encode a false claim, an accusation, a hypothesis, a question, or a fictional statement. A more precise formulation is:

> The packet is the canonical meaning. The carrier is the lens through which that meaning is authored, inspected, rendered, or signaled.

---

5. Final Architecture

LSTP v0.1 is organized as a seven-layer architecture.

```
Layer 0: Semantic Kernel
Layer 1: Lattice Text
Layer 2: Semantic Graph
Layer 3: Context Stack
Layer 4: Multimodal Interface
Layer 5: AI-Native Compression
Layer 6: Safety and Permissions
```

5.1 Layer 0: Semantic Kernel

The semantic kernel contains the canonical packet. It is the source of meaning.

The packet contains:

```
id
version
pragmatics
atoms
relations
context
confidence
permissions
evidence
```

```
output
carrier
audit
```

The core model is the **Octad Packet**:

1. Pragmatics
2. Atoms
3. Relations
4. Context
5. Confidence
6. Permissions
7. Evidence
8. Output

5.2 Layer 1: Lattice Text

Lattice text is the human-readable, typable, copyable, and versionable representation.

Example:

```
!analyze @sales[-4Q] :: cmp(region) + detect(anomaly) + infer(cause~)
{mode=readonly, assume:market_stable}
-> brief@z2
%0.82
; ctx↑0
```

The text layer should be treated as the first implementation target.

5.3 Layer 2: Semantic Graph

The semantic graph represents entities and relations as nodes and edges.

Example:

```
{
  "nodes": [
    {"id": "sales", "type": "target"},
    {"id": "cmp_region", "type": "operation"},
    {"id": "detect_anomaly", "type": "operation"},
    {"id": "infer_cause", "type": "operation"},
    {"id": "brief", "type": "output"}
  ],
  "edges": [
    {"from": "sales", "to": "cmp_region", "type": "apply"},
    {"from": "sales", "to": "detect_anomaly", "type": "apply"},
```

```
    {"from": "detect_anomaly", "to": "infer_cause", "type": "supports"},
    {"from": "infer_cause", "to": "brief", "type": "produce"}
  ]
}
```

This layer is suited for reasoning, planning, causal models, multi-agent exchange, and visual rendering.

5.4 Layer 3: Context Stack

The context stack reduces repetition by allowing compact references to prior frames.

Examples:

```
ctx.push{domain=startup, metric=activation, period=-3M}
?bottleneck ↑0 -> answer@z2
```

Multi-agent context references are also supported:

```
?risk ↑2@agent_risk -> summary@z2
```

This means:

> Refer to the frame two steps up in the context stack associated with `agent_risk`.

The context stack is essential for long-running conversations, agent swarms, and iterative design workflows.

5.5 Layer 4: Multimodal Interface

The multimodal layer includes radial UI, spatial authoring, audio signals, haptics, and other carriers.

Radial UI should be treated as both:

1. a rendering interface; and
2. an authoring interface.

A user may construct a packet spatially by selecting a target, adding operation nodes, assigning constraints, choosing output format, and setting permission state. That visual construction compiles into the same canonical packet as Lattice text.

Audio should be treated as a signal layer, not a full reasoning carrier. It can encode urgency, confidence, permission state, completion, error, or attention requirement.

5.6 Layer 5: AI-Native Compression

AI-native compression includes vector pointers, embeddings, fingerprints, and latent retrieval mechanisms.

However, these are assistive, not authoritative.

Correct use:

```
symbolic graph + optional vector pointers
```

Incorrect use:

```
pure vector exchange as the only meaning layer
```

Vectors may help with similarity and retrieval, but symbolic structure is required for auditability, safety, and cross-system interoperability.

5.7 Layer 6: Safety and Permissions

Any packet capable of real-world effects must include a permission mode.

Core modes:

Mode	Meaning
`readonly`	Inspect only; no changes
`preview`	Prepare action but do not execute
`sandbox`	Execute only in isolated test context
`confirm`	Ask user before execution
`commit`	Apply real-world change
`auto`	Execute automatically under prior authorization
`advisory`	Provide recommendation only
`reply_or_explain_only`	Only respond or explain

Safety principle:

> No irreversible real-world action may occur without explicit permission.

---

6. Protocol Overview

6.1 Canonical Octad Packet

The Octad Packet is the canonical representation of a message.

```
{
  "id": "pkt_example_001",
  "version": "0.1",
  "pragmatics": {},
  "atoms": {},
  "relations": [],
  "context": {},
  "confidence": {},
  "permissions": {},
  "evidence": {},
  "output": {},
  "carrier": {},
  "audit": {}
}
```

The fields are:

Field	Purpose
`pragmatics`	Why the packet exists and what communicative act it performs
`atoms`	Entities, concepts, documents, data, people, claims, or objects
`relations`	Typed links between atoms
`context`	Stack references, prior packets, variables, agent frames
`confidence`	Certainty, uncertainty, probability, or required confidence
`permissions`	What the packet is allowed to do
`evidence`	Evidence, assumptions, missing support, challenge paths
`output`	Desired response format and detail level

6.2 Lattice Text Syntax

General pattern:

```
<intent><action> @<target>[<scope>] :: <operations>
{constraints}
-> <output>@z<n>
```

```
%<confidence>
; metadata
```

Examples:

```
!summarize @meeting :: extract(decisions + owners + deadlines)
{mode=readonly}
-> action_items@z2
```

```
?cause @churn↑ :: infer(cause~)
{assume:pricing_constant}
-> hypotheses@z3
%0.7
```

```
!email @client :: propose_meeting
{mode=preview, tone=warm+firm}
-> draft@z2
```

6.3 Operators

Symbol	Meaning	Example
`!`	Command or request	`!analyze`
`?`	Question	`?cause`
`.`	Statement or context declaration	`.ctx{...}`
`!!`	Urgent command or alert	`!!alert`
`@`	Entity, target, or source	`@sales`
`::`	Begins operation block	`:: cmp(region)`
`+`	Combine or include	`risk + cost`
`-`	Exclude	`-jargon`
`->`	Desired output	`-> brief`
`{}`	Constraints, assumptions, permissions	`{mode=readonly}`
`[]`	Scope, range, time, filter	`[-4Q]`
`%`	Confidence	`%0.82`
`~`	Approximate or uncertain	`cause~`
`#`	Macro	`#RCA`
`@z<n>`	Zoom level	`@z3`

Symbol	Meaning	Example
`↑n`	Context stack reference	`↑2`
`↑n@agent`	Agent-specific context reference	`↑2@agent_risk`

6.4 Zoom Semantics

Zoom controls semantic depth, not merely response length.

Zoom	Meaning
`@z0`	Signal only
`@z1`	Minimal answer
`@z2`	Concise useful answer
`@z3`	Working detail
`@z4`	Deep analysis
`@z5`	Full reasoning structure
`@zmax`	Exhaustive expansion, if available

Example:

```
?best @strategy :: eval(risk + upside + effort) -> recommendation@z2
```

requests a concise recommendation.

```
?best @strategy :: eval(risk + upside + effort) -> recommendation@z5
```

requests a deeper analysis with assumptions, evidence, alternatives, and tradeoffs.

6.5 Macros

Macros compress common structures.

Definition:

```
def #Decision = {
  options + criteria + tradeoffs + reversibility + risks + recommendation
}
```

Use:

```
!decide @job_offer :: #Decision -> answer@z3
```

Macros may be:

- built-in;
- user-defined;
- domain-specific;
- organization-specific;
- versioned.

Macro expansion must be inspectable.

6.6 Ambiguity

Ambiguity should be represented explicitly, not hidden.

Example:

```
ambig{
  "bank": finance%0.62 | river_edge%0.38
}
```

Clarification request:

```
?clarify @bank:[finance|river_edge]
```

This prevents the system from silently choosing one interpretation where uncertainty matters.

6.7 Evidence and Auditability

Evidence should be attached where claims require support.

Example:

```
claim: churn↑ <= onboarding_friction↑ %0.74
because[evidence:support_tickets + funnel_drop + user_interviews]
assume[pricing_constant]
challenge[check cohort_by_channel]
```

This does not guarantee truth. It makes the support structure inspectable.

LSTP should not claim to eliminate hallucination. A more rigorous claim is:

> LSTP makes unsupported certainty structurally visible.

7. Carrier Model

LSTP distinguishes between full encodings, structured renderings, and signal carriers.

7.1 Full Encoding Carriers

These can preserve the full packet:

- Lattice text
- JSON packet
- Semantic graph

7.2 Structured Rendering Carriers

These can represent most of the structure but may hide detail behind zoom or interaction:

- radial UI
- spatial AR interface
- visual glyph system

7.3 Signal or Pointer Carriers

These usually do not preserve the full packet. They point to it or summarize its state:

- audio chord
- haptic signal
- status light
- small notification

7.4 Assistive Carriers

These support retrieval, similarity, and compression, but must not replace symbolic structure:

- embeddings
- vector references
- semantic fingerprints
- latent memory pointers

This distinction prevents unrealistic claims of lossless translation across all modalities.

---

8. Safety Model

The safety model is central to LSTP.

8.1 Permission Requirement

Any packet that may cause an external side effect must include an explicit permission mode.

Examples of side effects include:

- sending a message;
- changing a file;
- committing code;
- purchasing an item;
- deleting data;
- scheduling an event;
- invoking another agent with execution authority.

Safe packet:

```
!email @client :: propose_meeting
{mode=preview}
-> draft@z2
```

Potentially dangerous packet:

```
!email @client :: negotiate_price
{mode=auto}
-> sent
```

The second requires prior explicit authorization.

8.2 Permission Modes

Mode	Allowed behavior
`readonly`	Read, inspect, summarize, analyze
`preview`	Prepare output but do not execute
`sandbox`	Test in an isolated environment
`confirm`	Ask before executing
`commit`	Apply a real change
`auto`	Execute automatically under pre-approved policy
`advisory`	Recommend only
`reply_or_explain_only`	Only answer or explain

8.3 Audit Trail

Packets should support audit fields:

```
{
  "audit": {
    "created_by": "user",
    "created_at": "timestamp",
    "source_carrier": "lattice_text",
    "validated": true,
    "schema_version": "0.1",
    "revision": 1
  }
}
```

Audit metadata is especially important for multi-agent systems and real-world actions.

8.4 Challengeability

Claims should be challengeable.

Example:

```
challenge[
  verify_source,
  check_counterexample,
  test_alternative_hypothesis
]
```

This allows a user, agent, or validator to inspect the basis of a claim rather than accepting compressed output as authoritative.

---

9. Worked Example

Natural language:

> The message is incorrect! The dog is actually blue. Why did you lie about it?

Lattice text:

```
!correct @message : incorrect!
!state @dog.color = blue
?reason @you :: lie_about(@claim)
{tone=frustrated+direct, mode=reply_or_explain_only}
-> explanation@z2
; ctx↑1
```

Canonical packet:

```
{
  "id": "pkt_correct_message_001",
  "version": "0.1",
  "pragmatics": {
    "intent": "question",
    "action": "request_reason",
    "force": ["correction", "accusation", "question"],
    "tone": "frustrated_direct",
    "urgency": "high"
  },
  "atoms": {
    "message": "@message",
    "dog": "@dog",
    "you": "@you",
    "claim": "dog_color"
  },
  "relations": [
    {
      "type": "assert",
      "subject": "@message",
      "predicate": "is_incorrect"
    },
    {
      "type": "corrective_claim",
      "subject": "@dog",
      "predicate": "color",
      "object": "blue"
    },
    {
      "type": "accuse",
      "subject": "@you",
      "predicate": "lied_about",
      "object": "@claim"
    },
    {
      "type": "request_reason",
      "subject": "@you",
      "object": "lied_about_it"
    }
  ],
  "context": {
    "prior_message_ref": "ctx↑1"
  },
  "confidence": {
    "type": "speaker",
    "value": 0.95
  },
  "permissions": {
    "mode": "reply_or_explain_only"
  },
```

```
  "evidence": {
    "provided": [],
    "needed": ["source_or_image_showing_dog_color"]
  },
  "output": {
    "format": "explanation",
    "zoom": 2
  },
  "carrier": {
    "preferred": "radial"
  },
  "audit": {
    "source": "manual_example"
  }
}
```

Radial UI interpretation:

```
Center:
  correction + accusation + question

Inner nodes:
  @message
  @dog
  @you

Relation nodes:
  message is incorrect
  dog.color = blue
  you lied_about claim
  request reason

Outer context:
  frustrated tone
  high urgency
  prior message reference
  explanation requested
```

This example demonstrates how a socially loaded natural-language statement can be decomposed into explicit semantic components without claiming that the accusation is factually true.

---

10. Implementation Roadmap

The first implementation should be spec-first and text-first. Radial UI, audio, and AI-native carriers should come after the packet schema and parser are stable.

Phase 1: Specification and Schema

Deliverables:

- `spec/lstp-v0.1.md`
- `spec/octad-schema.json`
- `spec/grammar.ebnf`
- `spec/operator-table.md`
- `spec/permissions-safety.md`
- examples in `.lat` and `.packet.json`

Goal:

> Make the protocol understandable, reviewable, and testable.

Phase 2: Text Parser

Deliverables:

- tokenizer;
- parser;
- packet compiler;
- JSON schema validator;
- CLI command;
- valid and invalid fixtures;
- conformance tests.

Example command:

```
lstp parse examples/lattice/analyze-sales.lat
```

Goal:

> Convert Lattice text into canonical Octad packet JSON.

Phase 3: Context Stack and Macros

Deliverables:

- `ctx.push`;
- `ctx.pop`;
- `↑n`;
- `↑n@agent`;
- variable references;
- macro definitions;
- macro expansion;
- macro inspection.

Goal:

Reduce repetition while preserving clarity and auditability.

Phase 4: Semantic Graph Compiler

Deliverables:

- packet-to-graph compiler;
- node and edge typing;
- graph JSON export;
- graph validation;
- examples for causal chains, product decisions, and multi-agent delegation.

Goal:

Enable machine reasoning and AI-to-AI exchange through symbolic graph structures.

Phase 5: Radial UI Prototype

Deliverables:

- 2D radial renderer;
- packet-to-radial mapping;
- radial-to-packet editing;
- zoom interaction;
- context depth visualization;
- permission and confidence visual markers.

Goal:

Demonstrate bidirectional visual authoring and visualization.

Phase 6: Audio and Haptic Signal Layer

Deliverables:

- mapping from urgency, confidence, and permission state to audio/haptic signals;
- packet pointer mechanism;
- screen-free status notification prototype.

Goal:

Use audio and haptics for attention and status, not full reasoning.

Phase 7: AI-Native Exchange

Deliverables:

- symbolic graph packet exchange format;
- vector attachment specification;
- semantic fingerprinting;
- multi-agent context references;
- audit logging.

Goal:

> Allow AI agents to exchange structured, inspectable semantic packets with optional vector support.

Phase 8: Natural Language Bridge

Deliverables:

- natural language to Lattice translator;
- Lattice to natural language expander;
- ambiguity detection;
- permission inference warnings;
- human confirmation flow.

Goal:

> Allow ordinary users to interact naturally while preserving packet structure underneath.

---

11. Limitations and Open Questions

LSTP v0.1 remains a draft. Several questions require further design and testing.

11.1 Usability

It is not yet known whether ordinary users will prefer writing Lattice text directly, using natural language translated into Lattice, or using visual authoring tools.

11.2 Grammar Complexity

The text grammar must balance compactness with readability. A grammar that is too expressive may become difficult to parse and teach.

11.3 Ontology Alignment

AI-to-AI packet exchange requires shared meaning for atoms, relations, and operations. Different systems may interpret the same symbolic terms differently unless ontologies are versioned and explicit.

11.4 Vector Interoperability

Vector references are model-dependent. LSTP must avoid assuming that embeddings from different systems are directly interchangeable.

11.5 Security

A compact protocol could make harmful or unintended actions easier to trigger if permission models are weak. Safety rules and validation must be part of the core protocol.

11.6 Evidence Quality

A packet can require evidence fields, but it cannot guarantee that the evidence is valid, complete, or correctly interpreted. External validation remains necessary.

11.7 Multimodal Fidelity

Text, JSON, and graph can fully encode packets. Radial UI can render complex structure but may hide detail. Audio and haptic signals are mostly pointer or status carriers. This distinction must remain explicit.

---

12. Conclusion

LSTP proposes a semantic transport layer for the AI era. It separates meaning from medium by representing communication as canonical, inspectable packets that can be rendered through text, graph, radial UI, audio signals, and AI-native structures.

Its core contribution is not a new alphabet or a purely visual language. Its contribution is a structured packet model that makes intent, entities, relations, context, confidence, permissions, evidence, and output requirements explicit.

The mature design principle is:

> One meaning. Many carriers. Always inspectable.

The practical implementation path is clear:

1. define the specification;
2. implement the Octad schema;
3. build the Lattice text parser;
4. add validation and conformance examples;
5. implement context and macros;
6. compile packets into graphs;
7. prototype radial UI;
8. add audio and AI-native carriers.

LSTP does not claim to solve ambiguity, hallucination, or communication bandwidth completely. It proposes a rigorous architecture for reducing ambiguity, improving recoverability, increasing information density, and making AI-mediated communication safer and more inspectable.

Appendix A: Minimal Example Set

A.1 Summarization

```
!summarize @meeting :: extract(decisions + owners + deadlines)
{mode=readonly}
-> action_items@z2
```

A.2 Analysis

```
!analyze @sales[-4Q] :: cmp(region) + detect(anomaly) + infer(cause~)
{mode=readonly, assume:market_stable}
-> brief@z2
%0.82
```

A.3 Decision

```
!decide @product_launch[next_Q] :: eval(demand + readiness + legal_risk +
support_burden)
{risk_tolerance=med, mode=advisory}
-> recommendation@z4
```

A.4 Context

```
ctx.push{domain=ecommerce, metric=conversion_rate, period=-2M}
?cause @drop ↑0 -> hypotheses@z3
```

A.5 Multi-Agent Context

```
?risk ↑2@agent_risk -> summary@z2
```

A.6 Macro

```
def #Decision = {
  options + criteria + tradeoffs + reversibility + risks + recommendation
}

!decide @job_offer :: #Decision -> answer@z3
```

A.7 Permission

```
!email @client :: propose_meeting
{mode=preview, tone=warm+firm}
-> draft@z2
```

A.8 Evidence

```
claim: churn↑ <= onboarding_friction↑ %0.74
because[evidence:support_tickets + funnel_drop + user_interviews]
assume[pricing_constant]
challenge[check cohort_by_channel]
```

Appendix B: Recommended Repository Placement

This file should be placed at:

```
docs/whitepaper.md
```

Recommended related files:

```
spec/lstp-v0.1.md
spec/octad-schema.json
spec/grammar.ebnf
spec/operator-table.md
spec/permissions-safety.md
examples/lattice/
examples/packets/
docs/architecture.md
docs/security-model.md
docs/implementation-roadmap.md
```